TreasuryAudit360 – How It Was Built

Step-by-step build record: n8n automation, Claude AI, web app, GitHub package and Android app

---

Workflow automation for audit of the Treasury department of a bank in India

Regulatory coverage: RBI | SEBI | IFSCA (GIFT City) | FEMA

Auditing framework: ICAI Standards on Auditing and Standards on Internal Audit

Platform: n8n Cloud | Claude (Anthropic) | Google Sheets | Gmail | Lovable

Capstone project submitted by: **CA Ravishankar Balakrishnan Aiyar**

Membership No.: **040524**

Batch: **AIL2B105**

Date: **30 October 2026**

Contents

1. Overview of what was built 3
2. Planning decisions 3
3. Building the data store (n8n Workflow 0 – Setup) 4
4. Building the Audit Engine (n8n Workflow 1) 4
5. Building the API (n8n Workflow 2) 5
6. Building the hosted web app with Lovable AI 5
7. Building the standalone app for GitHub 5
8. Making it an installable Android web app (PWA) 6
9. Building the native Android app (APK and AAB) 6
9.1 Toolchain installed (with the owner's approval of the Android SDK licence) 6
9.2 Android project (Android_App\project) 6
9.3 Build pipeline (Android_App\build.ps1, PowerShell, no Android Studio needed) 6
9.4 Problems solved 7
9.5 Result 7
10. Installing and publishing on Android 7
11. Documents and demo video 7
12. Final folder layout (Desktop\TreasuryAudit360) 7

This document records, in the order it was done, every step used to build TreasuryAudit360, with the tool, language and output of each step. All bank data is fictitious.

1. Overview of what was built

#	Component	Tool / platform	Language	Output
1	Data store with fictitious treasury data	Google Sheets (created by n8n)	JavaScript (seed code)	Workbook "Treasury Audit - Demo Bank (Capstone)" with 9 tabs
2	Audit engine: 23 tests, AI report, findings register, audit trail, email	n8n Cloud	TypeScript (n8n Workflow SDK), JavaScript, n8n expressions, JSON	Workflow "Treasury Audit - 1 Audit Engine" (published webhook)
3	API for the app: findings, data, management responses	n8n Cloud	TypeScript, JavaScript	Workflow "Treasury Audit - 2 Findings & Data API" (3 webhooks)
4	AI report drafting	Claude Sonnet 5.5 (Anthropic node in n8n)	English system prompt + JSON	Audit Committee report in HTML
5	Hosted web app (first version)	Lovable AI	React, TypeScript, Tailwind CSS, shadcn/ui, Recharts	https://treasuryaudit360.lovable.app
6	Standalone web app for GitHub	Claude Code	HTML, CSS, JavaScript	`index.html` (single file, no build step)
7	Installable Android web app (PWA)	Claude Code	JSON manifest, JavaScript service worker, Python (icons)	`manifest.webmanifest`, `sw.js`, icons, screenshots
8	Native Android app	Android SDK (API 36), JDK 17	Java, Android XML	`TreasuryAudit360.apk` (signed), `TreasuryAudit360.aab`
9	Documents and demo video	Python, Microsoft Word, PowerPoint	Python, PowerShell, Markdown	Guide (Word/PDF), narration script, 34-slide video (MP4)

AI tools used:

- **Claude Opus 5.5** in Claude Code (Claude desktop app) designed and built every component. It was connected to n8n and Lovable through MCP connectors.
- **Claude Sonnet 5.5** writes the audit report inside n8n on every run.
- **Lovable AI** generated and hosts the React version of the app.

Every audit finding is decided by deterministic JavaScript rules. The AI only writes the narrative.

2. Planning decisions

1. **Scope agreed:** domestic treasury (RBI), forex and derivatives (RBI/FEMA), capital markets (SEBI), and the GIFT City IFSC Banking Unit (IFSCA), all aligned to the ICAI Standards on Auditing.
2. **Platform choices:**
   - n8n Cloud for automation.
   - Google Sheets as the data store, so examiners can inspect it.
   - Claude for AI drafting.
   - Lovable for the first hosted front end.
3. **Design principle:** all regulatory thresholds are stored as **parameters** in the Google Sheet, not hard-coded. When RBI, SEBI or IFSCA change a limit, the auditor edits one cell.
4. **Audit approach:** 100% population testing using **CAAT** (Computer-Assisted Audit Techniques), plus a top-5 walkthrough sample for vouching (SA 530 / SA 500).

3. Building the data store (n8n Workflow 0 – Setup)

1. In n8n, the existing credentials were reused: Google Sheets OAuth2, Gmail OAuth2 and Anthropic (n8n AI credits).
2. A workflow was written in **TypeScript** using the n8n Workflow SDK:
   - **Manual Trigger → Google Sheets: Create spreadsheet** with 9 tabs: Parameters, Deals, Investments, Counterparties, Dealers, Positions, Restricted_List, Findings, Audit_Runs.
   - 7 **Code nodes (JavaScript)**, one per tab, each returning the fictitious rows (26 deals, 15 investments, 13 counterparties, 9 dealers, 7 daily positions, 2 restricted scrips, 18 parameters).
   - 7 **Google Sheets: Append row** nodes. Cell format is *RAW*, so dates and times are stored as text.
3. The workflow was validated, created and run once.
   - The first run failed because Google rejected the locale "en_IN". The locale option was removed and the second run succeeded.
4. **Designed exceptions:** the data deliberately contains 31 exceptions across all 23 tests. Examples:
   - a dealer confirming their own deal
   - an off-market G-Sec deal
   - a NOOP breach
   - a trade in a restricted-list share
   - an INR transaction in the IBU

4. Building the Audit Engine (n8n Workflow 1)

Step	Node	Type / language	What it does
1	POST Run Audit	Webhook	Path `treasury-audit/run`, responds through a Respond node, CORS *
2	Daily Concurrent Audit 7pm	Schedule trigger (disabled)	Optional continuous concurrent audit
3	Normalize Request	Set node (n8n expressions)	Reads auditor name, email, engagement with defaults
4	Read 7 tabs	Google Sheets (Execute Once)	Parameters, Deals, Investments, Counterparties, Dealers, Positions, Restricted list
5	Run Audit Tests	Code node – **JavaScript**	23 tests, risk rating, compliance matrix, ICAI SA list, SA 530 sample, data fingerprint, JSON brief for AI
6	Draft Audit Report with Claude	Anthropic node – **Claude Sonnet 5.5**	System prompt: senior CA, SIA 370 / SA 265 structure, 8 sections, HTML only, "use only the facts"
7	Assemble Report	Code node – JavaScript + HTML	Cleans the AI output, appends Annexure A (findings) and B (tests), builds API response
8	Prepare Findings Rows → Save Findings Register	Code + Google Sheets append	One row per finding with status "Open"
9	Build Audit Trail Record → Log Audit Trail (SA 230)	Set + Google Sheets append	Run id, preparer, reviewer, rating, counts, data fingerprint
10	Email Report to Auditor	Gmail (HTML)	Emails the report if an email address is given
11	Respond with Results	Respond to Webhook (JSON)	Returns everything to the web app

Issues found and fixed during testing:

1. **Claude returned "Bad request".** The temperature setting conflicts with the model, so it was removed.
2. **The report was cut off.** Adaptive thinking used part of the 6,000-token budget. The limit was raised to 14,000 and the prompt asks for under 1,100 words.

3. **Error tolerance:** the AI, email and sheet-writing steps are set to *continue on error*. An AI or email failure never stops the audit, and the report falls back to the system-generated annexures.

Test results:

- The live webhook returned 31 findings (21 High, 10 Medium) in about 75 seconds.
- All 23 tests produced their designed exceptions.

5. Building the API (n8n Workflow 2)

1. **GET `/treasury-audit/findings`:** reads Findings and Audit_Runs, then a JavaScript node picks the latest run (or `?run_id=`) and builds summaries by risk, regulator and status.
2. **GET `/treasury-audit/data`:** reads all 7 data tabs and returns them as one JSON object for the Data Explorer.
3. **POST `/treasury-audit/response`:** normalises the body, then **Google Sheets: Update row** matching `finding_id` writes status, management response, owner, target date and timestamp.
4. Both workflows were **published**. CORS was tested from a browser origin (preflight returned 204 with the correct headers).

6. Building the hosted web app with Lovable AI

1. A Lovable project was created through the Lovable MCP connector. One detailed prompt specified the 7 pages, the 4 API endpoints and their JSON shapes, Indian number formatting and the navy/gold style.
2. Lovable generated a React + TypeScript + Tailwind + shadcn/ui app with Recharts charts.
3. It was published to https://treasuryaudit360.lovable.app and tested end to end: dashboard, live audit run, report, findings, compliance and data.

7. Building the standalone app for GitHub

1. A single `index.html` was written by hand in **HTML, CSS and plain JavaScript**. It has no frameworks and no build step, so it runs on any web host or straight from disk.
2. **Pages:**
   - Dashboard (with CSS bar charts and a donut chart)
   - Run Audit (animated stages)
   - Audit Report (print and download)
   - Findings Register (filters, CSV export, management response side panel)
   - Compliance Matrix
   - Data Explorer
   - About
   - Settings, to point the app at any n8n instance
3. **Exports for the repository:**
   - The 3 n8n workflows were exported as importable JSON files. The private Sheet ID was replaced with `YOUR_GOOGLE_SHEET_ID` and the credential IDs were removed.
   - The sample data was exported as CSV files.
4. **Tested** against the live back end through a local web server: a full audit run, a management response saved and confirmed in the register, a phone-sized layout, opening from disk, and no console errors.

8. Making it an installable Android web app (PWA)

1. **Icons:** generated with **Python (Pillow)**. A gold bank building on navy, at 192 and 512 px, in "maskable" versions for Android adaptive icons and an Apple touch icon.
2. **Manifest (`manifest.webmanifest`):** name, short name, theme colours, standalone display, icons, phone and desktop screenshots, and shortcuts to "Run Audit" and "Findings Register".
3. **Service worker (`sw.js`, JavaScript):** caches the app shell so the app opens offline. Audit data is never cached.
4. An **Install app** button was added for Android Chrome.
5. **Verified with the browser engine's own checks:** "installable", 0 manifest errors, and the app opens offline with a clear message when data cannot load.

9. Building the native Android app (APK and AAB)

9.1 Toolchain installed (with the owner's approval of the Android SDK licence)

Tool	Source	Purpose
Microsoft OpenJDK 17	aka.ms (Microsoft)	Java compiler, keytool, jarsigner
Android SDK command-line tools	dl.google.com (Google)	sdkmanager
Android platform API 36 (Android 16)	Google	`android.jar` to compile against
Build-tools 36.1.0	Google	aapt2, d8, zipalign, apksigner
bundletool	Google (GitHub)	Builds the Play Store bundle (AAB)

Everything is installed in `C:\Users\ravia\AndroidBuildTools`. Only the main Android SDK licence was kept accepted.

9.2 Android project (Android_App\project)

File	Language	Purpose
`AndroidManifest.xml`	Android XML	Package `com.treasuryaudit360.app`, version 1.0.0, min Android 7.0 (API 24), target Android 16 (API 36), Internet permission
`src/.../MainActivity.java`	**Java**	Full-screen WebView. Serves the bundled web app from a private HTTPS origin, opens external links in the browser, and handles the back button (including Android 16 predictive back)
Bridge in `MainActivity.java`	Java ↔ JavaScript	`print()` uses Android's print service (Save as PDF); `saveFile()` saves CSV and HTML to Downloads/TreasuryAudit360
`res/values*/`	Android XML	App name, navy/gold colours, theme, Android 12+ splash screen
`res/mipmap-*`	PNG (Python-generated)	Adaptive, themed (monochrome) and legacy launcher icons at 5 screen densities
`assets/www/`	HTML/JS	The web app, copied in at build time

9.3 Build pipeline (Android_App\build.ps1, PowerShell, no Android Studio needed)

1. Copy the web app into `assets/www`.
2. **aapt2 compile** the resources, then **aapt2 link** them into the base APK (and a protobuf version for the AAB).
3. **javac** compiles the Java against `android.jar`.
4. **d8** converts the Java bytecode to Android's Dalvik format (`classes.dex`).
5. Generate a 4096-bit RSA **upload signing key** (`signing\treasuryaudit360-upload.jks`, valid 27 years) with a random password stored in `signing\keystore-password.txt`.
6. Package the APK, run **zipalign** (16 KB page alignment, required by Google Play), then **apksigner** signs it (v2 + v3 schemes) and verifies it.
7. **bundletool** builds the AAB, then **jarsigner** signs it and bundletool validates it.

8. Print a summary (package, versions, permissions).

To rebuild after changing the web app, run this command in the `Android_App` folder:

```
powershell -ExecutionPolicy Bypass -File build.ps1
```

9.4 Problems solved

1. **The SDK licence prompt didn't receive the answer.** It was run through Java directly, with the answers fed from a file.
2. **zipalign reported "header mismatch".** Appending to the ZIP caused it, so the package is now rewritten fresh. The result is 0 warnings and alignment verified.
3. **Android 16 changes the back button** for apps targeting API 36. A modern back handler (`OnBackInvokedCallback`) was added.

9.5 Result

- `output\TreasuryAudit360.apk`: install directly on phones. Signed, verified, about 0.1 MB.
- `output\TreasuryAudit360.aab`: upload to the Google Play Console.

10. Installing and publishing on Android

1. **Install the APK on a phone:** copy `TreasuryAudit360.apk` to the phone (WhatsApp to yourself, Google Drive or USB) and tap it. Allow "Install unknown apps" when asked, then tap **Install**.
2. **Google Play:**
   - Create a Play Console account and click **Create app**.
   - Upload `TreasuryAudit360.aab` under **Production → Create release**.
   - Use the icons and phone screenshots from `GitHub_Repo\TreasuryAudit360\icons` and `\screenshots`.
   - Complete Data safety, Content rating and a privacy policy.
   - New personal accounts must run a closed test before production.
3. **Keep the signing key safe.** `Android_App\signing\` holds the key and its password. Back it up privately and never upload it to GitHub. Every future update must be signed with the same key. Google Play App Signing can also hold the final release key.

11. Documents and demo video

1. **Guide:** written in Markdown, converted to Word with **Python (python-docx)**, then exported to PDF with Microsoft Word.
2. **Screenshots:** a headless **Microsoft Edge** browser was driven through the Chrome DevTools Protocol from **PowerShell** to capture full-HD screens of every page, including a live audit run.
3. **Slides:** 1920×1080 images built with **Python (Pillow)**. They include tech labels on every screen, workflow diagrams, a code slide and a prompt slide.
4. **Video: PowerPoint** (automated through COM) assembled the slides with fade transitions and timings and exported a 1080p MP4. Speaker notes contain the narration.

12. Final folder layout (Desktop\TreasuryAudit360)

Folder / file	Contents
`Treasury_Audit_Automation_Guide.docx / .pdf / .md`	Main project guide

Folder / file	Contents
How_TreasuryAudit360_Was_Built.docx	This build record
Demo_Video\	MP4 video, PowerPoint deck, narration script, voice recording guide
GitHub_Repo\TreasuryAudit360\	Upload-ready website (PWA), n8n workflow JSON, sample data, docs
TreasuryAudit360_GitHub_Ready.zip	The same repository as one ZIP
Android_App\output\	TreasuryAudit360.apk, TreasuryAudit360.aab
Android_App\project\, build.ps1	Android source code and the build script
Android_App\signing\	**Private** signing key and password. Back up and never share